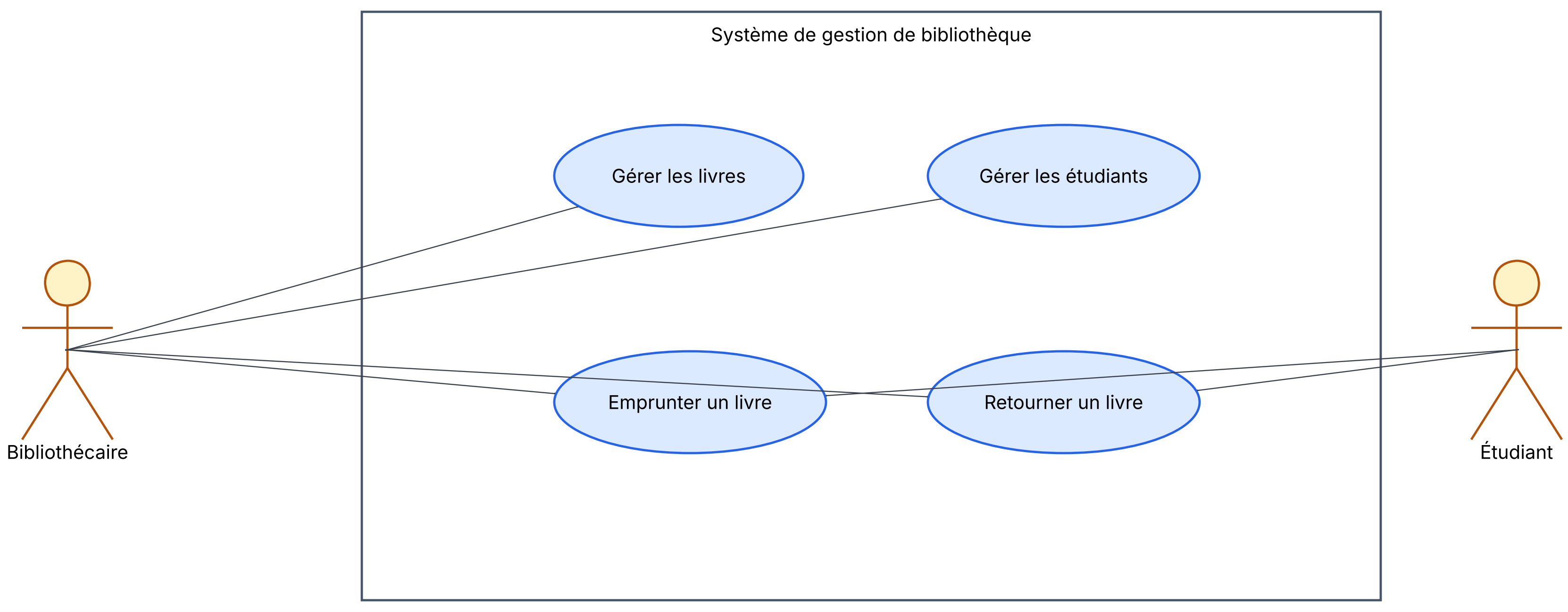

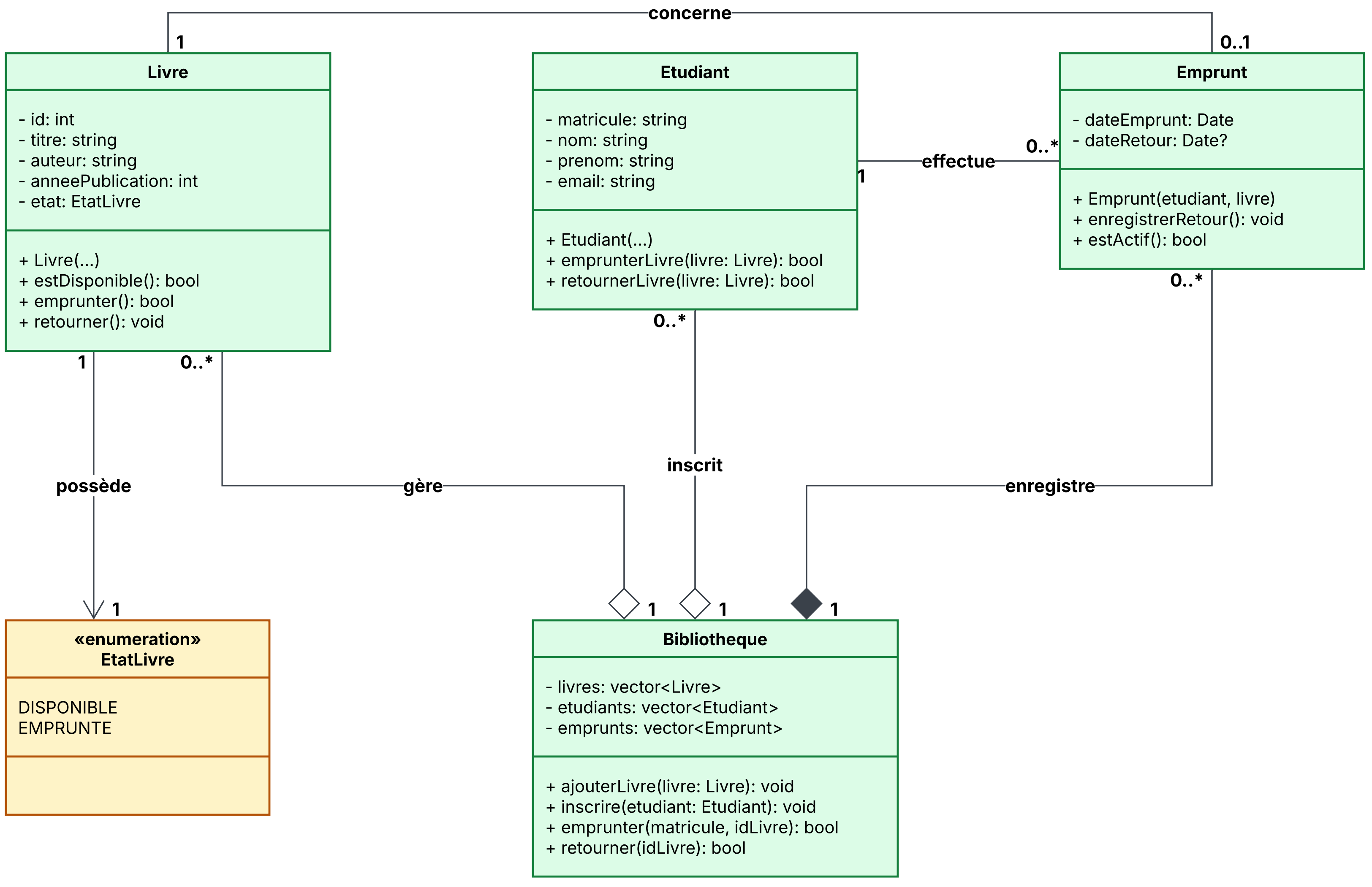

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <algorithm>

using namespace std;

enum class EtatLivre { DISPONIBLE, EMPRUNTE };

class Livre {
private:
    int id;
    string titre, auteur;
    int anneePublication;
    EtatLivre etat;
public:
    Livre(int id, string titre, string auteur, int annee)
        : id(id), titre(titre), auteur(auteur),
          anneePublication(annee), etat(EtatLivre::DISPONIBLE) {}

    int getId() const { return id; }
    bool estDisponible() const { return etat == EtatLivre::DISPONIBLE; }
    bool emprunter() {
        if (!estDisponible()) return false;
        etat = EtatLivre::EMPRUNTE;
        return true;
    }
    void retourner() { etat = EtatLivre::DISPONIBLE; }
};

class Etudiant {
private:
    string matricule, nom, prenom, email;
public:
    Etudiant(string mat, string nom, string prenom, string email)
        : matricule(mat), nom(nom), prenom(prenom), email(email) {}

    string getMatricule() const { return matricule; }
};

class Emprunt {
private:
    Etudiant* etudiant;
    Livre* livre;
    bool actif;
public:
    Emprunt(Etudiant* e, Livre* l) : etudiant(e), livre(l), actif(true) {}
    void enregistrerRetour() { actif = false; livre->retourner(); }
    bool estActif() const { return actif; }
};

class Bibliotheque {
private:
    vector<Livre> livres;
    vector<Etudiant> etudiants;
    vector<Emprunt> emprunts;
public:
    void ajouterLivre(const Livre& livre) { livres.push_back(livre); }
    void inscrire(const Etudiant& etudiant) { etudiants.push_back(etudiant); }

    bool emprunter(const string& matricule, int idLivre) {
        auto it = find_if(livres.begin(), livres.end(),
            [idLivre](const Livre& l) { return l.getId() == idLivre; });
        if (it == livres.end() || !it->emprunter()) return false;
        auto et = find_if(etudiants.begin(), etudiants.end(),
            [&matricule](const Etudiant& e) { return e.getMatricule() == matricule; });
        if (et == etudiants.end()) { it->retourner(); return false; }
        emprunts.emplace_back(&*et, &*it);
        return true;
    }

    bool retourner(int idLivre) {
        for (auto& e : emprunts)
            if (e.estActif() && e.concerneLivre(idLivre)) { e.enregistrerRetour(); return true; }
        return false;
    }
};

int main() {
    Bibliotheque b;
    b.ajouterLivre(Livre(1, "UML et C++", "Auteur", 2024));
    b.inscrire(Etudiant("E001", "Nom", "Prenom", "email@univ.sn"));
    cout << (b.emprunter("E001", 1) ? "Emprunt reussi\n" : "Emprunt refuse\n");
    cout << (b.retourner(1) ? "Retour reussi\n" : "Retour refuse\n");
}

// Remarque : retourner(idLivre) doit sélectionner l’emprunt actif correspondant à idLivre.
```